Ingeniería en Sistemas de Información y Ciencias de la Computación.
Curso: Programación III
Sección: B
Ingeniero: David Álvarez

Tarea IV
Lista Simple

Erix Alejandro Solares Flores
Carné: 9941-20-23978

```cpp
#include <iostream>

using namespace std;

struct Nodo {
    int dato;
    Nodo* siguiente;
};

int main() {
    Nodo* cabeza = NULL;
    Nodo* actual = NULL;

    int valor_ingresado;
    char continuar = 's';

    while (continuar == 's' || continuar == 'S') {

        cout << "Ingrese un numero: ";
        cin >> valor_ingresado;

        Nodo* nuevo_nodo = new Nodo();
        nuevo_nodo->dato = valor_ingresado;
        nuevo_nodo->siguiente = NULL;

        if (cabeza == NULL) {
            cabeza = nuevo_nodo;
        } else {

            actual = cabeza;
            while (actual->siguiente != NULL) {
```

main

```
10      int main() {
17          while (continuar == 's' || continuar == 'S') {
26              if (cabeza == NULL) {
28              } else {
29
30                  actual = cabeza;
31                  while (actual->siguiente != NULL) {
32                      actual = actual->siguiente;
33                  }
34                  actual->siguiente = nuevo_nodo;
35              }
36
37              cout << "Deseas ingresar otro numero? (s/n): ";
38              cin >> continuar;
39          }
40
41          int contador_nodos = 0;
42          int suma_total = 0;
43          float promedio = 0;
44
45          cout << "\n--- Valores en la lista ---" << endl;
46          actual = cabeza;
47          while (actual != NULL) {
48              cout << "Valor: " << actual->dato  << endl;
49
50              suma_total += actual->dato;
51              contador_nodos++;
52
53              actual = actual->siguiente;
54          }
55
```

f main

```
10  int main() {
47      while (actual != NULL) {
48          cout << "Valor: " << actual->dato  << endl;
49
50          suma_total += actual->dato;
51          contador_nodos++;
52
53          actual = actual->siguiente;
54      }
55
56      cout << "\nRepresentacion visual:" << endl;
57      actual = cabeza;
58      while (actual != NULL) {
59          cout << actual->dato << " -> ";
60          actual = actual->siguiente;
61      }
62      cout << "NULL" << endl;
63
64      if (contador_nodos > 0) {
65          promedio = (float)suma_total / contador_nodos;
66      }
67
68      cout << "\n--- Resultados Finales ---" << endl;
69      cout << "Total de nodos creados: " << contador_nodos <<  endl;
70      cout << "Suma de los valores: " << suma_total <<  endl;
71      cout << "Promedio de los valores: " << promedio <<  endl;
72      cout << "Erix Alejandro Solres Flores";
73
74      return 0;
75  }
```

f main

```
"C:\Users\chaud\OneDrive\Escritorio\UMG 1 2026\Programacion III\Hoja_de_trabajo_lista_enlazada\Ejercicio.exe"
Ingrese un numero:55

Deseas ingresar otro numero? (s/n):s

Ingrese un numero:64

Deseas ingresar otro numero? (s/n):s

Ingrese un numero:94

Deseas ingresar otro numero? (s/n):n


--- Valores en la lista ---
Valor: 55
Valor: 64
Valor: 94

Representacion visual:
55 -> 64 -> 94 -> NULL

--- Resultados Finales ---
Total de nodos creados: 3
Suma de los valores: 213
Promedio de los valores: 71
Erix Alejandro Solres Flores
Process finished with exit code 0
```

Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas

Nombre:	Erix Alejandro Solares Flores	Carné:	9941-20-23178

Carrera	Ingeniería en Sistemas	Cod. Carrera	9941
Asignatura	Programación III	Cod. Curso	022
Ciclo	Quinto Ciclo	Jornada	Domingos
Catedrático	Ing. MBA David Alvarez	Fecha	08/03/2026
Semestre	Primer Semestre	Sección	B

Objetivo de aprendizaje

El estudiante será capaz de crear dinámicamente una lista enlazada utilizando memoria dinámica, permitiendo que el usuario agregue nodos indefinidamente hasta que decida detener el proceso.

Planteamiento del problema

- Un sistema debe almacenar números ingresados por el usuario en una lista enlazada dinámica.
- El programa no debe conocer previamente cuántos elementos se almacenarán.
- El usuario podrá seguir ingresando valores hasta que decida detener el proceso.
- Cada número ingresado debe almacenarse en un nodo creado dinámicamente en memoria y agregarse al final de la lista.

Requisitos del programa

El programa debe cumplir con las siguientes condiciones:

1. Definir una estructura Nodo que contenga:
   - Un valor entero.
   - Un puntero al siguiente nodo.
2. Utilizar memoria dinámica (new) para crear cada nodo.
3. Utilizar un ciclo while para permitir el ingreso de datos indefinidamente.
4. El usuario debe decidir si desea seguir ingresando datos.
5. Cada nuevo nodo debe agregarse al final de la lista.
6. Al finalizar el ingreso de datos, el programa debe recorrer la lista y mostrar todos los valores almacenados.

Estructura base sugerida

Comenzar con la siguiente estructura:

```
#include <iostream>
using namespace std;

struct Nodo {
    int dato;
    Nodo* siguiente;
};

int main() {

    Nodo* head = NULL;
    Nodo* actual = NULL;

    int valor;
    char opcion = 's';

    while(opcion == 's') {

        cout << "Ingrese un numero: ";
        cin >> valor;

        // Crear nodo dinámicamente

        // Insertar nodo en la lista

        cout << "Desea ingresar otro numero? (s/n): ";
        cin >> opcion;
    }

    // Recorrer lista

    return 0;
}
```

Ejemplo de ejecución esperada

```
Ingrese un numero: 10
Desea ingresar otro número? (s/n): s

Ingrese un numero: 7
Desea ingresar otro número? (s/n): s

Ingrese un numero: 25
Desea ingresar otro número? (s/n): n

Lista generada:

10 -> 7 -> 25 -> NULL
```

Preguntas de análisis

Responda las siguientes preguntas después de ejecutar el programa.

1. ¿Qué función cumple el puntero actual?
2. ¿Qué ocurriría si se elimina la línea que asigna nuevo->siguiente = NULL?
3. Explique cómo se conectan los nodos dentro de la lista.

+ 1 Punto Extra

Modifique el programa para que también calcule:

- La cantidad total de nodos creados
- La suma de los valores almacenados
- El promedio de los valores

Escaneado con CamScanner